CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 10: Hash Tables: Linear Probing, Chaining, and Double Hashing

Practical Aim :

To implement hash tables using different collision handling strategies and develop the ability to reason about why collisions occur, how different strategies resolve them, and what trade-offs each approach involves.

Problem 1 :

A small parking lot has exactly 10 numbered slots (0 to 9). When a vehicle arrives, its

registration number is used to assign it a slot by taking the last digit of the number. If that slot is

already taken, the vehicle moves to the next available slot in order, wrapping around if

necessary. Given a sequence of vehicle registrations, assign each to a slot using this rule and

display the final state of all ten slots.

Describe what your solution does when it encounters an already occupied slot. What happens

when the parking lot is nearly full and many vehicles keep landing on the same slot? Can you

think of a situation where your solution might loop indefinitely?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush> cd "c:\Users\kaush\OneDrive\Desktop\code\DSA\" ; if ($?) { g++
Probing.cpp -o pr101Linear_Probing } ; if ($?) { .\pr101Linear_Probing }
Enter number of vehicles (max 10): 3
Enter vehicle registration numbers:
12
25
49

Parking Slots:
0 : Empty
1 : Empty
2 : 12
3 : Empty
4 : Empty
5 : 25
6 : Empty
7 : Empty
8 : Empty
9 : 49
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Outcome/Conclusion:

Learned to implement a hash table using linear probing to handle collisions.

Problem: Faced:

Handling collisions and finding the next available slot in the hash table.

Problem 2 :

A library uses a system with 10 shelves numbered 0 to 9. Each book's code is used to decide

which shelf it belongs to by taking the code modulo 10. Unlike the parking lot, each shelf can

hold multiple books stacked together — when two books map to the same shelf, they are simply

added to that shelf's stack. Given a sequence of book codes, place each on the correct shelf

and display the final contents of all shelves.

Describe how your solution handles two books that map to the same shelf. What happens to

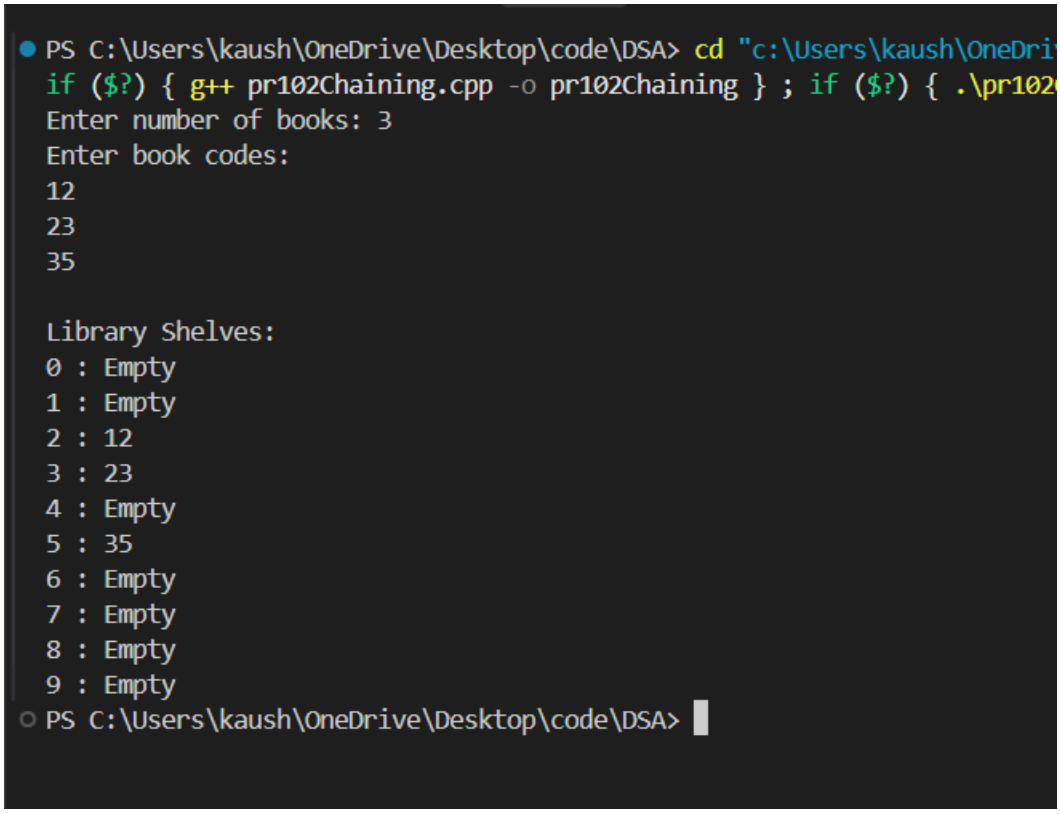
search time on a shelf as more and more books pile onto it? Can you think of what would cause

all books to end up on the same shelf?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Outcome/Conclusion:

Learned to implement a hash table using chaining to handle collisions.

ProblemFaced:

Storing multiple book codes at the same shelf and managing collisions.

Problem 3 :

A university assigns student IDs to storage slots using a two-step rule. The first rule determines

the initial slot. If that slot is taken, instead of simply moving one slot forward, a second rule

based on the ID itself determines how many slots to jump. This means different IDs that collide

will probe different sequences of slots, spreading them more evenly. Given a sequence of

student IDs, assign each to a slot using both rules and display the final state of the table.

Describe how your solution decides how many slots to jump when a collision occurs. What is

the risk if the jump size happens to be a factor of the table size? How does this strategy differ

from Problem 10.1 in how it spreads collisions?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:

```
                                                cd "c:\Users\kaush
if ($?) { g++ pr103Double_Hashing.cpp -o pr103Double_Hashing }
ng }
Enter number of student IDs (max 10): 3
Enter student IDs:
23
1
2

Hash Table:
0 : Empty
1 : 1
2 : 2
3 : 23
4 : Empty
5 : Empty
6 : Empty
7 : Empty
8 : Empty
9 : Empty
```

Outcome/Conclusion:

Learned to implement double hashing to resolve collisions using two hash functions.

Problem Faced:

Calculating the jump size and finding an empty slot when collisions occur.

Key Questions / Analysis / Interpretation

Q1. If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy.

Answer: The first vehicle occupies its hashed slot, and the remaining vehicles occupy consecutive slots. This causes clustering and makes future insertions slower.

Q2. Suppose all book codes are multiples of 10. Predict the final shelf layout and explain why.

Answer: All books will be stored on shelf 0 because every multiple of 10 has a remainder of 0 when divided by 10. This increases search time on that shelf.

Q3. What would happen if the second hash function always produced a jump size of zero? Explain why.

Answer: The algorithm repeatedly checks the same slot and cannot resolve collisions. Therefore, the second hash function must produce a suitable non-zero jump size.